Introduction to OpenCL (Week 1 – 4)

The aim of these workshops is to introduce you to practical aspects of parallel programming using the OpenCL framework. There are 4 tutorial sessions for this module in total, each designed to take approximately 2 weeks or 4 hours to complete, except tutorial two which should be completed over 4 weeks.

Each session is meant to serve as a tutorial for a particular topic in parallel programming with explanations and practical tasks that should help you to understand the theory covered during lectures. These tasks are not assessed but are designed in a way that should lead you to the solutions required in the practical assessment for this module.

To make the most out of these sessions, discuss your answers with your colleagues and/or ask demonstrators for feedback. If you need a refresher on C++, check out one of the online tutorials (e.g. http://www.cplusplus.com/doc/tutorial) covering basic I/O, vectors, strings, etc.

1. Installing OpenCL (Week 1)

We have provided steps in Appendix A of this tutorial to help you install OpenCL. These instructions have been tested on the Windows platform and are fully functional.

2. OpenCL Concepts (Week 1 and 2)

There are several key concepts that form the basis of the OpenCL framework. This part of the tutorial is used to illustrate how the main OpenCL concepts: platforms, devices, contexts, and queues fit together in practice.

Platform

The OpenCL **platform** is simply a specific implementation of the standard by individual hardware vendors. On a single computer there might be several OpenCL platforms/implementations (e.g. CPU supported by the Intel platform and GPU by the AMD platforms). Below is a code snippet that retrieves all available OpenCL platforms using cl::Platform::get(), checks if any exist, and then iterates through them to print each platform's name, vendor, version, and profile using getInfo<>().

```
std::vector<cl::Platform> platforms;
cl::Platform::get(&platforms);

if (platforms.empty()) {
    std::cout << "No OpenCL
    platforms found\n"; return 1;
}

for (auto &p : platforms) {
    std::cout << "============================\n";
    std::cout << "Platform Name:    " <<
    p.getInfo<CL_PLATFORM_NAME>() << "\n"; std::cout << "Platform
    Vendor:                    " <<
    p.getInfo<CL_PLATFORM_VENDOR>() << "\n"; std::cout <<
    "Platform Version: " << p.getInfo<CL_PLATFORM_VERSION>() <<
    "\n"; std::cout << "Profile:    " <<
    p.getInfo<CL_PLATFORM_PROFILE>() << "\n";
}
```

Devices, Compute Units, and Processing Elements

A "**device**" is an actual hardware device supporting OpenCL. A "**device**" will run its code on "**computing units**" (e.g. a single core of a modern CPU or an NVIDIA's GPU streaming multiprocessor), which in turn consists of "**processing elements**" (e.g. ALU in CPU, an NVIDIA's GPU streaming processor) (see Fig. 2).

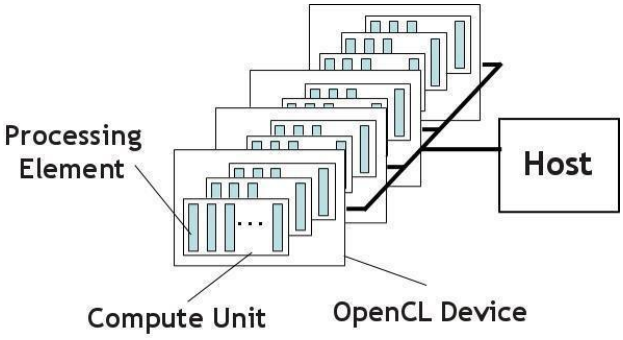

*Figure 2: Each OpenCL **device** consists of at least one **Compute Unit**, which in turn contains one or more **Processing Elements** (after https://github.com/HandsOnOpenCL).*

The **code snippet below retrieves all OpenCL devices** on a platform using p.getDevices(CL_DEVICE_TYPE_ALL, &devices), checks if any exist, and then prints key device details — such as name, vendor, maximum compute units, and global memory size — using getInfo<>().

```
std::vector<cl::Device> devices;
p.getDevices(CL_DEVICE_TYPE_ALL,
&devices);

if (devices.empty()) {
    std::cout << "  No devices found for this platform.\n";
} else {
    for (auto &d : devices) {
        std::cout << "------------------------- \n";
        std::cout << "  Device Name:      " <<
        d.getInfo<CL_DEVICE_NAME>() << "\n"; std::cout << "Device
        Vendor:         " << d.getInfo<CL_DEVICE_VENDOR>() << "\n";
        std::cout << "  Max Compute Units: " <<
        d.getInfo<CL_DEVICE_MAX_COMPUTE_UNITS>() << "\n"; std::cout << "Global Mem
        Size:           " << d.getInfo<CL_DEVICE_GLOBAL_MEM_SIZE>() / (1024 * 1024)
    }
}
```

Summary of OpenCL Key Concepts

Below is a summary of the key concepts we have covered so far, except for context and queues, which we will explore in later examples.

Term	Description	Example
Platform	Vendor's OpenCL implementation	Intel OpenCL SDK, AMD ROCm
Device	Hardware capable of executing kernels	CPU, GPU, Accelerator
Compute Unit (CU)	Execution block	CPU core, GPU multiprocessor
Processing Element (PE)	Smallest execution unit	ALU, shader core
Context	Environment containing devices	Group of devices
Queue	Interface for host-device commands	Kernel/memory operations

Tasks 1.1 Exploring OpenCL Devices

Use the concepts learnt so far to complete the code in "01_list_devices.cpp". You code should list all platforms and their GPU devices on your system and display the following information for each:

- For Platform (Platform name, Vendor, and Version)
- For Devices (Device name, Maximum compute units, Maximum work-group size, and Maximum clock frequency)

The codes templates have been well commented with TODOs to support you in completing your codes. Please read through the definition of key terms listed below which should help your understanding to complete this tasks.

- **Device Name:** The readable name identifying the specific OpenCL device (e.g., GPU or CPU).
- **Maximum Compute Units:** The total number of independent processing cores available for parallel execution.
- **Maximum Work-Group Size:** The largest number of work-items (threads) that can execute together in a single work-group.
- **Maximum Clock Frequency:** The highest speed, in MHz, at which each compute unit operates.

Code	Purpose
cl::Platform::get(&platforms);	Queries all available OpenCL platforms (e.g., NVIDIA, AMD, Intel).
platform.getDevices(CL_DEVICE_TYPE_GPU, &devices);	Gets all GPU devices for each platform.
device.getInfo<CL_DEVICE_NAME>()	Retrieves the device name string.
device.getInfo<CL_DEVICE_MAX_COMPUTE_UNITS>()	Returns the number of compute units (like cores).
device.getInfo<CL_DEVICE_MAX_WORK_GROUP_SIZE>()	Maximum local work-group size supported.
device.getInfo<CL_DEVICE_MAX_CLOCK_FREQUENCY>()	Device base clock in MHz.

For discussion with colleagues, compare the specs of different CPU and GPU devices. Which device do you expect to perform better in parallel tasks?

3. Basic Profiling (Week 2)

You will explore some more concepts in this part of the tutorial. OpenCL enables the measurement of execution time (i.e. **profiling**) using "**events**". The events are attached to different **queue** commands and collect information about the timing of the respective commands. A **context** serves as a container that manages all OpenCL resources associated with a device, such as memory buffers, program objects, kernels, and command queues. It defines the environment within which kernels execute and memory objects are allocated, ensuring that operations are coordinated and executed correctly on the chosen device. You have been provided some starter code templates (see the folder "`02_basic_profiling.cpp`" in Tutorial 1 folder) with some TODOs that you must complete to understand the profiling concept of OpenCL. Below are also some key methods required to complete the task.

1. The line `cl::Context context(device);` creates an OpenCL context for the selected device.
   ```
   cl::Context context(device);
   ```
2. Enable profiling for the queue:
   ```
   cl::CommandQueue queue(context, CL_QUEUE_PROFILING_ENABLE);
   ```
3. Create an event and attach it to a queue command responsible for the kernel launch:
   ```
   cl::Event prof_event;
   queue.enqueueNDRangeKernel(kernel_add, cl::NullRange,
   cl::NDRange(vector_elements), cl::NullRange, nullptr,
   &prof_event);
   queue.finish();
   ```
4. Display the kernel execution time at the end of the program:
   ```
   std::cout << "Kernel execution time [ns]:" <<
   prof_event.getProfilingInfo<CL_PROFILING_COMMAND_END>() -
   prof_event.getProfilingInfo<CL_PROFILING_COMMAND_START>() <<
   std::endl;
   ```

Task 2.1: Complete all the TODOs in the starter code template that will enable you to start checking the kernel execution time for different devices for specific program task.

You can get full information about each profiling event including enqueueing and preparation time by using the provided helper function `GetFullProfilingInfo()`. The function accepts two parameters: an event and time unit format. For example, to print out the detailed breakdown of our event in microseconds add the following line of code:

```
std::cout << GetFullProfilingInfo(prof_event, ProfilingResolution::PROF_US)
<< std::endl;
```

4. Large datasets (Week 3)

It should be rather difficult to see any significant difference between execution time on different devices when the input vectors are very short, because fixed overheads (queueing commands, launching the kernel, and copying memory) dominate the runtime. To make performance differences measurable, we will increase the vector length and include both computation and data transfer time in profiling. You are provided with a template OpenCL program (`03_large_datasets.cpp`) that already selects platforms and devices, creates the OpenCL context and command queue (with profiling enabled), and builds a simple vector-add kernel. Your task is not to write the full program, but to complete the TODO sections to test larger input vectors, attach profiling events, compute elapsed times, and compare CPU and GPU performance.

Chunking support is included in the template. The program queries device memory limits (CL_DEVICE_MAX_MEM_ALLOC_SIZE and CL_DEVICE_GLOBAL_MEM_SIZE) and uses them to compute a safe chunk size so that very large vectors can be processed in segments. Instead of allocating device buffers for the full vector length, the code allocates chunk-sized buffers once and then repeats the sequence "upload → kernel → download" until the entire vector has been processed. This ensures the program still works even when N is larger than what the device can allocate in a single buffer.

Task 1: Profile your code for different vector lengths on a CPU and GPU device. The template already contains the input vectors; you only need to vary N to test different dataset sizes (e.g.,

10K, 100K, 1M). Run the program multiple times on a CPU device and a GPU device and record the total time.

```
const int N = 1000;   // Try: 10'000, 100'000, 1'000'000
std::vector<int> A(N, 1);
std::vector<int> B(N, 2);
std::vector<int> C(N, 0);
```

Task 2: Attach events for profiling (transfers + kernel). Profiling is enabled on the command queue, but timing is only available if you attach events to the operations being measured. In the chunking loop, attach events to:

A. Profile Input Uploads (Host → Device). Attach events to the buffer uploads for input vectors A and B. These events will later be used to measure data transfer time.

```
queue.enqueueWriteBuffer(bufferA, CL_TRUE, 0, thisBytes, A.data() +
offset, nullptr, &A_event);
queue.enqueueWriteBuffer(bufferB, CL_TRUE, 0, thisBytes, B.data() +
offset, nullptr, &B_event);
```

B. Profile Kernel Execution. Attach an event to the kernel enqueue command. This event measures how long the kernel runs on the device.

```
queue.enqueueNDRangeKernel(kernel_add, cl::NullRange,
cl::NDRange(thisChunkElems), cl::NullRange,  nullptr, &kernel_event);
```

C. Profile Output Download (Device → Host). Attach an event to the read-back of the output buffer C.

```
queue.enqueueReadBuffer(bufferC, CL_TRUE, 0, thisBytes, C.data() +
offset, nullptr, &C_event);

queue.finish();
```

NOTE: offset is the starting index of the current chunk in the host vectors (A, B, C). It moves forward by chunkElems each iteration to process the next segment. **thisBytes** is the size (in bytes) of the current chunk being transferred, computed as thisChunkElems * sizeof(int). Details of these are in the code template.

Task 3: Compute Elapsed Times from Events. Events store profiling timestamps, but you must compute elapsed time manually.

A. Define a Helper Function. Create a lambda function that returns elapsed time (in nanoseconds) from an OpenCL event.

```
auto duration_ns = [](cl::Event &e) {
    return e.getProfilingInfo<CL_PROFILING_COMMAND_END>() -
           e.getProfilingInfo<CL_PROFILING_COMMAND_START>();
};
```

B. Calculate Transfer and Execution Times. Convert the measured durations from nanoseconds to milliseconds and compute the time for each operation. The total operation time accounts for both computation and data movement by accumulating the timings across all chunks:

```
uploadA_ms_total   += duration_ns(A_event)      * 1e-6;
uploadB_ms_total   += duration_ns(B_event)      * 1e-6;
```

```
exec_ms_total      += duration_ns(kernel_event) * 1e-6;
downloadC_ms_total += duration_ns(C_event)      * 1e-6;
```

C. The total operation time should include both transfers and computation:

```
double total_ms = uploadA_ms_total + uploadB_ms_total + exec_ms_total +
downloadC_ms_total;
```

What you should observe: For small vector sizes, CPU and GPU execution times are typically similar because overhead dominates. As the vector length N increases, the GPU performance improves relative to the CPU due to increased parallelism. With larger datasets, the time spent transferring data between the host and the device becomes a significant part of the total runtime. A GPU provides a clear performance advantage only when the computational workload is large enough to outweigh the data transfer overhead.

5. Kernels (Week 4)

So far what we have been doing is embedding the kernel code inside the .cpp file. However, separating the kernel code from the .cpp file allows kernels to be modified or reused without recompiling the host code and reflects standard best practice in real-world OpenCL applications. In these tasks we have provided you with a separate file "`04_kernels/my_kernels.cl`" which has a single kernel function called "`add`" and a template source file (`04_kernels.cpp`) to complete all the TODOs. The kernel file current has only a parallel addition codes:

```
kernel void add(global const int* A, global const int* B, global int* C)
{
     int id = get_global_id(0);
     C[id] = A[id] + B[id];
}
```

Open the "`kernels/my_kernels.cl`". file, you can see that this is an ordinary C function with some extra qualifiers. The `kernel` label indicates that this is an OpenCL kernel function, and the `global` label indicates that the input parameters are in the main memory of the device. The `add` function takes pointers of two input integer vectors A and B (since we only read them, there is a `const` qualifier in front) and a pointer to output vector C where we store the result of the addition operation.

The parallel `add` consists of a single line of code that adds individual elements of vectors A and B and stores the sum in a corresponding element of vector C indicated by the `id` variable. As you can see, there is no for loop here, which would normally go through all elements of a vector. The kernel function is launched in parallel on as many computing units as available and as many times as necessary. Each launch gets a separate and unique ID which is obtained by `get_global_id(0)` function. The total number of kernel launches is equal to the vector length, which is specified as a parameter for the `enqueueNDRangeKernel` function: `cl::NDRange(vector_elements)`.

Task 1: Create a helper function to load the kernel source. Since the kernel code is stored in a separate `.cl` file, the host `.cpp` program must load that source code at runtime. This is handled by the `LoadKernelSource` helper function, which uses `std::ifstream` to open the kernel file, reads its entire contents into a `std::string`, and returns it. If the file cannot be opened, the function throws an error to prevent execution without valid kernel code.

```
std::string LoadKernelSource(const std::string &filePath) {
   std::ifstream file(filePath);
   if (!file.is_open()) throw std::runtime_error("Cannot open kernel file: " +
      filePath);
   return std::string(std::istreambuf_iterator<char>(file),
      std::istreambuf_iterator<char>());
}
```

Task 2: Implement multiple kernels

In this task we shall implement multiple kernels by adding our codes to the "`04_kernels/my_kernels.cl`".

A. **Multiple Kernels:** A single `.cl` file can contain multiple kernel functions. For example, in addition to an `add` kernel, you can define a separate `mult` kernel that performs parallel multiplication. On the host side, you can then choose which kernel to execute by creating a `cl::Kernel` object with the desired kernel name, such as `cl::Kernel(program, "mult")` or `cl::Kernel(program, "add")`.
B. **Single Kernel for Combined Operation:** In this task, implement an additional kernel called `multadd` within the same `.cl` file that computes `C=A×B+B` in a single pass. We will verify that the results match those obtained using the sequential multiplication and addition kernels the we will implement later. Then compare execution time on CPU and GPU devices and analyse which approach is more efficient for different vector sizes and devices.
C. **Add Kernel - Floating-Point Support:** To add floating-point support, update both the host and kernel code to use float instead of int. In the host code, change the input and output vectors to std::vector<float> and update all buffer size calculations to use sizeof(float). In the kernel file, define a new kernel function called addf that operates on float data using parameters global const float* A, global const float* B, and global float* C. The host should then select and launch the addf kernel in the same way as the integer version.
D. **Add Kernel - Double Precision Support:** Create a new kernel addd in the same .cl file to perform parallel addition on double vectors and update the host code to allocate std::vector<double> buffers and launch addd. Profile large vector sizes and compare CPU vs GPU performance for double versus int. Note that some devices do not support double precision, which can cause program build or runtime errors, and compare performance (and later results) across data types.
E. **Sequential Kernel Execution:** Modify the host code to execute two OpenCL kernels sequentially to compute `C=A×B+B`, without using the combined operation kernel written previously. Set up the multiplication kernel to write its result into buffer C using input buffers A and B. Then update the addition kernel so that it uses buffer_C as its first input argument, allowing the addition to operate on the intermediate result. Finally, enqueue the kernels in the correct order: launch the multiplication kernel first, followed by the addition kernel. The output buffer C should first contain the result of A × B and then be overwritten with the final result after the addition.

6. Additional tasks (Week 4)

The following tasks are designed for those eager students who finished all the tasks and

would like an extra challenge - the instructions provided are deliberately sparse! These tasks are optional and not necessary to progress further in the following tutorials. If you wish to implement your codes in C++ using visual studio code, then a starter

Task 1: Implement and profile a pure serial version of the vector equation: C = A * B + B. You can use any programming language you like for this task. You will need to use a loop and some system timing functions. Then compare the overall performance of the serial implementation to the parallel implementation run on CPU and GPU for different vector lengths (e.g. small, medium, large). Does the parallel solution bring any performance boost?

Task 2: Add an additional command line parameter to the OpenCL program performing the operation C = A * B + B which specifies the length of the vectors. This way, you will not need to recompile the program every time you want to change this value. Then collect the profiling info (the overall performance) for CPU and GPU devices and different size of vector length (at least 10 different values) and plot the values in a graph (e.g. in Excel or MATLAB). Repeat the same thing with the serial code.

7. Appendix A: Installing OpenCL

<u>Installation on Windows</u>

The following installation is based on windows only. You may need all the tools listed here to complete your installation and start using OpenCL.

1. **Visual Studio Code**
   - Download Visual Studio Code at: https://code.visualstudio.com/
   - Install the following extensions in VS Code:
     - **C/C++** (by Microsoft)
     - **C/C++ Compile Run** *(optional for quick testing)*
     - **Code Runner** *(optional)*
2. **Install OpenCL on Windows (Intel Hardware)**
   - Intel CPU OpenCL Runtime: Intel CPU Runtime for OpenCL is an OpenCL runtime and compiler that runs on Intel Core processor family or Intel Xeon processor family. Intel CPU Runtime for OpenCL is compatible with the OpenCL 3.0 standard specification and most extensions.
     - Download the Intel® CPU Runtime for OpenCL™ Applications (Windows installer) from Intel's website.
     - Run the .exe installer with Administrator privileges.
     - Intel® CPU Runtime for OpenCL™ Applications with SYCL* Support
   - Intel GPU Driver (if you have Intel graphics)
     - Intel Graphics Compute Runtime for OpenCL Driver is included with the Intel Graphics Driver package for Windows OS.
3. **NVIDIA CUDA Toolkit (for NVIDIA GPU)**
   - Download and Install the Toolkit. It is available at: https://developer.nvidia.com/cuda-toolkit

4. **Khronos OpenCL SDK**

Khronos provides an **official OpenCL SDK** including headers, libraries, and the ICD loader.

- Download the SDK: https://github.com/KhronosGroup/OpenCL-SDK/releases
- Extract it to a path, e.g.: **C:\OpenCL-SDK**
- Add to your environment variables: 1) setx OPENCL_SDK_DIR "C:\OpenCL-SDK" and 2) setx PATH "%PATH%;C:\OpenCL-SDK\bin"

5. **C++ compiler (Install MSYS2 and Toolchain)**

- Download & install MSYS2 available at: https://www.msys2.org/
- Open the MSYS2 UCRT64 terminal (not "MSYS" or "MINGW32").
- Update and install required packages:
  - Updates your **MSYS2 system** and all installed packages

    pacman -Syu
  - GCC / G++ compiler

    pacman -S mingw-w64-ucrt-x86_64-gcc
  - GDB debugger

    pacman -S mingw-w64-ucrt-x86_64-gdb
  - Make

    pacman -S mingw-w64-ucrt-x86_64-make
  - OpenCL headers and libraries

    pacman -S mingw-w64-ucrt-x86_64-opencl-headers mingw-w64-ucrt-x86_64-opencl-icd
  - Alternatively, you can run 2- 5 in a single command

    pacman -S mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-gdb mingw-w64-ucrt-x86_64-make mingw-w64-ucrt-x86_64-opencl-headers mingw-w64-ucrt-x86_64-opencl-icd

- Verify Install Paths - Your MSYS2 folder should now have the following files and folders:
  - C:\msys64\ucrt64\bin\g++.exe
  - C:\msys64\ucrt64\bin\gcc.exe
  - C:\msys64\ucrt64\bin\gdb.exe
  - C:\msys64\ucrt64\include\CL
  - C:\msys64\ucrt64\lib\libOpenCL.dll.a

6. **Add the Compiler's Binary Folder to the System PATH**

To ensure the system can locate the compiler and related tools from any command prompt, the binary (bin) directory must be added to the Windows PATH environment variable.

- Identify the correct bin folder for your toolchain.
  - In this case, it is installed in the folder: "C:\msys64\ucrt64\bin"
- Open Start, search Environment Variables, and open Edit the system environment variables. Click Environment Variables…. Under User variables click New and set MINGW_ROOT to C:\msys64\ucrt64. Then select Path, click Edit, click New, and add %MINGW_ROOT%\bin. Or C:\msys64\ucrt64\bin Confirm with OK on all windows and

reopen your terminal.

Installing on Linux

- Please follow the link below for details on how to install on Linux.
  - OpenCL-Guide/chapters/getting_started_linux.md at main · KhronosGroup/OpenCL-Guide